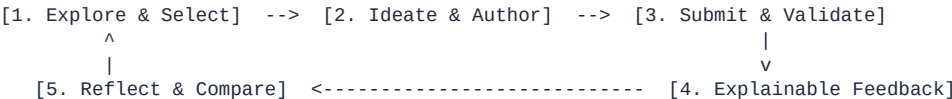


Design Note: Low-Level Design (LLD) Platform Architecture

Project: LLD Practice Platform | Author: Engineering Team | Focus: Domain Design, OOP Models, Evaluation Pipeline & Trade-offs

1. System Vision & The Learner Journey

The primary goal of the LLD Practice Platform is to provide a focused, low-friction, high-fidelity practice loop for software engineers sharpening their object-oriented modeling and machine coding skills. Unlike LMS platforms that emphasize passive video consumption, this system focuses on active, iterative design solving.



- 1. Explore & Select:** Learner browses 4 canonical problems (Parking Lot, Elevator Dispatcher, Vending Machine, Splitwise) with explicit functional requirements, constraints, and rubrics.
- 2. Ideate & Author:** In a tri-part studio, the learner authors class/interface code alongside design rationale notes, while an in-browser live Mermaid.js diagram visualizes class hierarchies in real time.
- 3. Submit & Validate:** Submissions enter an asynchronous state machine (PENDING → VALIDATING → ANALYZING → SYNTHESIZING → COMPLETED), preventing gateway timeouts.
- 4. Explainable Feedback:** Learner receives a 4-pillar scorecard (0–100), a SOLID compliance checklist, line-level refactoring nudges, and dynamic 'What-If' requirement stress tests.
- 5. Reflect & Compare:** Learner reviews versioned attempt history (v1, v2, v3), inspects score progression, and restores historical code with 1 click to refine.

2. Detailed Answers to Core Design Questions

Q1: What does a learner actually need to provide for an LLD practice attempt to be meaningful?

Raw executable code alone is insufficient for LLD because syntax cannot convey design intent or architectural trade-offs. Conversely, a pure hand-drawn diagram lacks verifiable contracts. Therefore, a meaningful LLD attempt requires a **Structured Tri-Part Submission**:

- 1. Core Domain Contracts (Code Skeleton):** Explicit class and interface declarations, field visibility, method signatures with typed contracts, and key method implementations showing entity collaboration.
- 2. Design Rationale & Trade-offs (Structured Notes):** Explicit justification of why specific patterns were selected (e.g. Strategy vs. State), boundary assumptions, and concurrency invariants.
- 3. Collaborative Walkthrough:** A short scenario simulation (e.g., sample driver flow) showing how client code interacts with the domain boundaries.

Q2: What makes feedback useful when there can be more than one valid LLD solution?

In LLD, there is never a single 'canonical' answer. A Parking Lot can legitimately use Strategy for pricing, Factory for spots, or Observer for gate displays. Useful feedback must be:

- Decoupled from Naming & Style Bias:** Never penalize a learner because they named a class `ParkingFloor` instead of `ParkingLevel`.
- Grounded in Universal Design Qualities:** Evaluates Single Responsibility (cohesion), Open/Closed (extensibility via polymorphism), Dependency Inversion (abstractions over concrete classes), and Invariant Defense (capacity guards, defensive errors).
- Socratic & Actionable:** Every critique provides (1) the architectural consequence, (2) a concrete refactoring nudge, and (3) a dynamic 'What-If' stress test.

Q3: Which parts of evaluation should be deterministic, and which parts benefit from an LLM?

We designed a **Dual-Track Hybrid Evaluation Pipeline** combining deterministic rigor with semantic intelligence:

Evaluation Tier	Evaluation Mechanism	Responsibilities & Capabilities
Tier 1: Deterministic Engine (AST & Rule Heuristics)	Fast, 100% reproducible, offline AST & regex structural parsing.	- Domain entity coverage check - God class detection (>6 methods) - Direct concrete coupling detection (new in methods) - Type-switching anti-pattern detection (<code>switch(type)</code>) - Defensive capacity & error invariant checks

Tier 2: Semantic Reasoning (LLM / Fallback Engine)	Context-aware semantic reasoning with strict 7s timeout & circuit breaker.	• Evaluates design intent & trade-off rationale • Assesses appropriateness of chosen design patterns • Synthesizes senior staff executive summaries • Generates dynamic downstream 'What-If' scenarios • Automatic fallback if offline/unconfigured
--	--	---

Q4: How would your design accommodate another evaluation approach or submission format later?

The backend strictly implements classic Gang of Four (GoF) design patterns to ensure open-ended extensibility:

- **Strategy Pattern** (`IEvaluationStrategy`): Pluggable evaluators (`StructuralEvaluator`, `SolidEvaluator`, `SemanticEvaluator`) share a unified interface. Adding a dynamic Docker test runner or security linter requires implementing `IEvaluationStrategy` and registering it in the pipeline without modifying existing classes.
- **Adapter Pattern** (`ISubmissionAdapter`): Translates alternative input formats (PlantUML text, JSON class specifications, Java source) into the canonical domain AST model.
- **Pipeline Pattern** (`EvaluationPipeline`): Chains validation, parsing, deterministic scoring, and semantic synthesis into a composable execution flow.

Q5: What should happen if evaluation takes time or fails?

- **Asynchronous Non-Blocking Processing**: Submissions immediately return HTTP 202 with an attempt ID and PENDING status. The client polls status updates with an animated progress stepper, avoiding gateway timeouts.
- **Circuit Breaker & Timeout Guard**: External LLM calls are wrapped in an `AbortSignal` with a strict 7-second timeout.
- **Graceful Degradation (Fallback Engine)**: If an LLM call fails, times out, or has no configured API key, the platform automatically triggers `FallbackEvaluator`. The learner receives complete, rubric-grounded feedback enriched with AST findings and domain 'What-If' challenges. *The system never hangs or crashes.*

3. Domain Model & Class Architecture

The domain layer follows Clean Object-Oriented Domain-Driven Design principles, decoupled from web frameworks and persistence mechanisms:

Domain Class / Interface	Design Pattern & Role	Key Responsibilities & Invariants
Problem	Aggregate Root (Entity)	Encapsulates problem metadata, functional requirements, constraints, rubric weights, starter templates, and benchmark solutions.
Attempt	Lifecycle Entity	Manages versioning (v1, v2), submission payload, and evaluation state transitions (PENDING → VALIDATING → ANALYZING → COMPLETED / FAILED).
IEvaluationStrategy	Strategy Pattern (Interface)	Common contract defining <code>evaluate(submission, problem, ast)</code> implemented by all evaluators.
StructuralEvaluator	Concrete Strategy	Evaluates domain entity completeness, detects God classes (>6 methods), and assesses interface-based abstraction.
SolidEvaluator	Concrete Strategy	Heuristic analysis for SRP (billing in lot), OCP (type switches), DIP (direct concrete new), ISP (fat interfaces), and capacity invariants.
EvaluationPipeline	Pipeline / Composite	Orchestrates validation, AST parsing, strategy execution, score weighting (0–100), and fallback synthesis.
IAttemptRepository	Repository Pattern	Decouples attempt persistence and version querying from storage mechanisms; supports in-memory thread-safe storage.

4. Key Architectural Trade-offs & Engineering Decisions

Decision Area	Chosen Approach	Alternative Considered	Engineering Rationale & Trade-off
System Boundary	Modular Monolith (Express API + Vite SPA)	Microservices with Kafka & Redis	For a 2-day LLD prototype, an in-process async worker avoids distributed systems overhead while keeping domain boundaries strictly decoupled.
Storage Engine	Thread-Safe In-Memory Repository with Seed Data	PostgreSQL / Prisma ORM	Allows instant zero-setup review by any engineer via <code>npm start</code> without database configuration hurdles.
Evaluation Engine	Deterministic AST + Semantic LLM Fallback	Pure LLM Prompts	Pure LLM is non-deterministic, hallucinates, and fails offline. AST heuristics guarantee 100% reproducible baseline scoring and zero-latency execution.

Live UML Diagram	Client-Side Dynamic Mermaid.js AST	Server-Side Java PlantUML jar	Runs securely in sandboxed client JS, has zero server latency, updates instantly as the user types, and avoids Java/Graphviz dependencies.
-------------------------	---------------------------------------	-------------------------------	--

5. Summary & Educational Conclusion

The LLD Practice Platform demonstrates how classic Object-Oriented Design principles (Strategy, Pipeline, Repository, and Adapter patterns) can be applied to solve the exact problem of LLD evaluation itself. By providing a structured practice studio, instant deterministic and semantic feedback, and iterative attempt versioning, the platform transforms ambiguous machine coding interviews into a measurable, repeatable path toward software engineering mastery.